

PROYECTO PRX

Informe del Sprint 1

Enlaces URL y tareas en el módulo de Notas

En este informe explicamos el trabajo que realizamos durante el Sprint 1 del mantenimiento del proyecto PRX. Nuestra tarea fue mejorar el módulo de Notas para que el usuario pueda agregar enlaces web y administrar una lista de tareas. También explicamos los archivos que modificamos, el código principal, los lenguajes utilizados, la calidad del resultado, los principios SOLID y los patrones de diseño que se aplicaron.

Dato	Información
Actividad	Presentación y defensa del Sprint 1
Fecha límite	23 de septiembre de 2026 a las 23:59
Proyecto	PRX Mantenimiento evolutivo
Módulo trabajado	Notas
Funcionalidades	Enlaces URL y tareas persistentes
Integrantes	JOSE HUAMANI HUAYPUNA VANIA GUARACHI RAMOS FERNANDO MARUPA ARNEZ CAMILA FATIMA RAMIREZ
Evaluación	Puntualidad Calidad SOLID Patrones

Resultado del Sprint

Al terminar el Sprint, una nota puede guardar enlaces válidos y tareas. Los enlaces se pueden abrir desde el detalle de la nota y las tareas se pueden marcar como completadas sin entrar nuevamente al formulario de edición. Los datos quedan guardados en MySQL y se recuperan después de recargar la página.

1 Resumen de nuestro trabajo

Nuestro trabajo consistió en ampliar una función que ya existía. Antes, una nota se concentraba principalmente en el título, el contenido y los archivos. En este Sprint incorporamos dos listas relacionadas con la nota: enlaces web y tareas. Para lograrlo tuvimos que realizar cambios tanto en Angular como en NestJS y Prisma.

1.1 Funciones realizadas

- Agregar uno o varios enlaces URL a una nota.
- Completar automáticamente HTTPS cuando el usuario no escribe el protocolo.
- Validar que los enlaces utilicen HTTP o HTTPS.
- Mostrar los enlaces guardados y abrirlos en una pestaña nueva.
- Agregar y eliminar tareas desde el detalle o la edición de la nota.
- Marcar o desmarcar tareas desde el modo de consulta.
- Guardar las tareas y enlaces como campos JSON mediante Prisma.
- Comprobar los permisos antes de modificar una nota.

1.2 Alcance del Sprint 1

En este Sprint presentamos únicamente los enlaces y las tareas de Notas. Las imágenes y el editor corresponden al siguiente Sprint, mientras que las mejoras de Bitácora se consideran otro incremento. Esta separación nos permite defender con claridad lo que realmente corresponde al Sprint 1.

2 Lenguajes y tecnologías utilizadas

Lenguaje o tecnología	Uso en nuestro trabajo	Motivo
TypeScript	Frontend Angular y backend NestJS	Permite trabajar con tipos y detectar errores de contrato.
HTML	Estructura de la pantalla de detalle	Muestra enlaces, tareas, botones y checkbox.
SCSS	Estilos de las páginas	Organiza la presentación visual y los estados de los controles.
Angular 21	Interfaz de Notas	Componentes, formularios, rutas y estado reactivo.
NestJS	API y casos de uso	Controladores, inyección de dependencias, DTO y CQRS.
Prisma	Persistencia	Acceso tipado a MySQL y manejo de campos JSON.
MySQL	Base de datos	Almacena las notas, tareas y enlaces.
JSON	Formato de las listas	Transporta y guarda tareas y enlaces relacionados con una nota.

3 Historias y tareas del Sprint

ID	Historia de usuario	Puntos	Estado
HU M01	Como usuario quiero agregar enlaces URL a una nota para relacionarla con recursos externos.	5	Hecho
HU M02	Como usuario quiero administrar tareas para controlar actividades pendientes y completadas.	8	Hecho

Tareas técnicas realizadas

Tarea	Trabajo realizado	Archivo o evidencia
T1	Crear modelos para tareas y enlaces	note model ts
T2	Agregar controles visuales	note detail HTML y TS
T3	Normalizar y validar URL	normalizeUrl e isValidUrl
T4	Enviar las listas al backend	note api ts
T5	Validar JSON y permisos	UpdateNoteHandler
T6	Guardar y recuperar listas	PrismaNoteRepository
T7	Agregar campos a la base de datos	schema prisma
T8	Comprobar compilación	ng build y nest build

4 Archivos donde están los cambios

Estas son las rutas que podemos mostrar durante la defensa. Están escritas desde la carpeta principal del proyecto.

4.1 Frontend

Archivo	Qué contiene
prx-frontend/src/app/features/notes/domain/models/note.model.ts	NoteTaskModel, NoteLinkModel y las listas de la nota.
prx-frontend/src/app/features/notes/presentation/pages/note-detail-page/note-detail-page.component.html	Secciones visuales de enlaces y tareas.
prx-frontend/src/app/features/notes/presentation/pages/note-detail-page/note-detail-page.component.ts	Agregar, eliminar y marcar tareas; agregar y validar enlaces.

Archivo	Qué contiene
prx-frontend/src/app/features/notes/domain/requests/update-note.request.ts	Contrato de actualización con tasks y links.
prx-frontend/src/app/features/notes/infrastructure/api/note.api.ts	FormData y envío HTTP al backend.

4.2 Backend y base de datos

Archivo	Qué contiene
prx-backend/src/modules/notes/domain/entities/note.entity.ts	NoteTask, NoteLink y contratos del dominio.
prx-backend/src/modules/notes/application/dto/requests/update-note-request.dto.ts	Entrada HTTP de tasks y links.
prx-backend/src/modules/notes/application/commands/update-note/update-note.handler.ts	Validación, permisos y actualización.
prx-backend/src/modules/notes/infrastructure/persistence/prisma-note.repository.ts	Lectura y escritura de JSON.
prx-backend/src/modules/notes/infrastructure/mappers/note-prisma.mapper.ts	Conversión de Prisma al dominio.
prx-backend/prisma/schema.prisma	Campos tasks y links del modelo Note.

5 Código principal que agregamos

5.1 Modelos de tareas y enlaces

```
export interface NoteTaskModel {
  id: string;
  title: string;
  completed: boolean;
}

export interface NoteLinkModel {
  id: string;
  url: string;
}
```

Este código está en note.model.ts. Lo utilizamos para establecer qué datos debe tener una tarea y qué datos debe tener un enlace. El identificador permite encontrar cada elemento y completed conserva el estado de la tarea.

5.2 Validación de enlaces

```
private normalizeUrl(value: string): string {
  const trimmed = value.trim();
  if (!trimmed) return '';
  return /^https?:\/\//i.test(trimmed) ? trimmed : `https://${trimmed}`;
}

private isValidUrl(value: string): boolean {
  try {
    const url = new URL(value);
    return url.protocol === 'http:' || url.protocol === 'https:';
  } catch {
    return false;
  }
}
```

normalizeUrl agrega HTTPS cuando hace falta. isValidUrl comprueba que la dirección tenga una estructura válida y que el protocolo sea HTTP o HTTPS. El backend vuelve a validar antes de guardar.

5.3 Marcar una tarea desde la vista

```
protected toggleTask(task: NoteTaskModel): void {
  if (!this.canEdit() || this.saving()) return;
  this.tasks.update((tasks) => tasks.map((item) =>
    item.id === task.id ? { ...item, completed: !item.completed } : item
  ));
  if (!this.editing()) this.persist(false);
}
```

Este método cambia el estado de una sola tarea. En modo Ver, persist false guarda el cambio sin abrir el editor. También se comprueba saving para evitar actualizaciones repetidas.

5.4 Envío al backend

```
formData.append('tasks', JSON.stringify(data.tasks));
formData.append('links', JSON.stringify(data.links));
formData.append('retainedFileIds', data.retainedFileIds.join(','));
```

Utilizamos FormData porque Notas también trabaja con archivos. Las tareas y los enlaces se convierten a JSON para enviarlos en la misma petición que los demás datos de la nota.

5.5 Validación y guardado en el backend

```
const tasks = this.parseTasks(command.dto.tasks);
const links = this.parseLinks(command.dto.links);

await this.noteRepository.update(command.id, {
  title: command.dto.title.trim(),
  content: command.dto.content.trim(),
  tasks,
  links,
  updatedBy: command.userId,
});
```

UpdateNoteHandler convierte el JSON, valida límites y comprueba permisos. Después llama al repositorio para guardar la nota. La lógica de negocio no escribe directamente en Prisma.

5.6 Campos de Prisma

```
model Note {
  tasks Json?
  links Json?
}
```

Estos campos se encuentran en prx-backend/prisma/schema.prisma. Son opcionales para mantener compatibilidad con notas antiguas que todavía no tienen tareas o enlaces.

6 Cómo funciona todo el recorrido

1. El usuario agrega una URL o cambia una tarea en Angular.
2. El componente valida los datos y actualiza el estado con Signals.
3. NoteFacade coordina la operación y NoteApi construye FormData.
4. NotesController recibe la petición y obtiene el usuario desde el JWT.
5. UpdateNoteHandler verifica permisos y valida las listas.
6. NoteRepository delega el guardado a PrismaNoteRepository.
7. Prisma guarda tasks y links como JSON en MySQL.
8. Los Mappers preparan la respuesta y Angular actualiza la pantalla.

7 Patrones de diseño utilizados

Patrón	Dónde lo usamos	Por qué lo usamos
CQRS	Commands para actualizar y Queries para consultar	Separa operaciones de lectura y escritura.
Repository	NoteRepository y PrismaNoteRepository	Evita que el caso de uso dependa directamente de Prisma.
Facade	NoteFacade	Simplifica la comunicación entre la página, la API y el Store.
Mapper	NotePrismaMapper y NoteResponseMapper	Convierte datos entre Prisma, dominio y respuesta.
DTO	UpdateNoteRequestDto	Define y valida lo que puede recibir el endpoint.
Store reactivo	Signals y NoteStore	Mantiene un estado controlado de datos, carga y errores.
Guard y Decorator	JWT y CurrentUser	Centraliza la identificación y autorización del usuario.
Soft delete	Campo status	Mantiene auditoría sin borrar inmediatamente el registro.

Explicación para la defensa

No utilizamos patrones solamente para mencionarlos. Repository separa la base de datos del caso de uso; Facade evita que la pantalla coordine todo; Mapper impide mezclar objetos de Prisma con la respuesta; DTO rechaza datos incorrectos y CQRS mantiene separadas las consultas de las operaciones que modifican información.

8 Principios SOLID

Principio	Aplicación en nuestro trabajo
Responsabilidad única	El componente maneja la interfaz, el Handler las reglas y el Repository los datos.
Abierto cerrado	Ampliamos las notas mediante contratos y componentes sin reemplazar toda la arquitectura.
Sustitución de Liskov	PrismaNoteRepository puede utilizarse donde se espera el contrato NoteRepository.
Segregación de interfaces	NoteApiContract contiene operaciones específicas del módulo de Notas.
Inversión de dependencias	UpdateNoteHandler depende de NoteRepository y no directamente de PrismaService.

La aplicación más clara de SOLID está en la separación de responsabilidades. No pusimos toda la lógica en el componente Angular. Cada capa realiza una parte concreta y se comunica mediante contratos.

9 Calidad del trabajo

9.1 Evidencias

- El frontend Angular compila y genera el bundle correctamente.
- El backend NestJS genera Prisma Client y compila sin errores.
- La URL se valida en frontend y backend.
- Los permisos impiden modificar notas sin acceso al repositorio.
- Los enlaces y el estado de las tareas permanecen después de recargar.
- Los enlaces externos se abren con noopener y noreferer.
- El estado saving evita enviar la misma operación varias veces.

9.2 Aspectos que todavía podemos mejorar

- Agregar pruebas automatizadas específicas para tareas, enlaces y permisos.
- Permitir agregar tareas durante la creación inicial; actualmente se administran desde detalle y edición.
- Reducir el archivo SCSS de create note page, que supera el presupuesto Angular por 818 bytes.

10 Puntualidad

La fecha límite indicada fue el 23 de septiembre de 2026 a las 23:59. Para respaldar la puntualidad debemos adjuntar o conservar el comprobante de entrega de la plataforma. El código y la documentación permiten demostrar el avance realizado, pero el registro de envío es la evidencia directa del cumplimiento de la fecha.

Evidencia	Estado
Historias de usuario	Implementadas
Compilación frontend	Correcta
Compilación backend	Correcta
Informe de respaldo	Preparado
Comprobante de entrega	Debe adjuntarse desde la plataforma

Conclusión

En este Sprint logramos que el módulo de Notas sea más útil para el seguimiento del trabajo. Los enlaces permiten relacionar recursos externos y las tareas permiten registrar actividades pendientes o completadas. La solución no quedó únicamente en la parte visual: incluye validación, permisos, casos de uso y persistencia. Además, la arquitectura existente permitió aplicar SOLID y patrones sin concentrar toda la lógica en una sola clase.

11 Cómo vamos a presentar

1. Explicaremos que el Sprint 1 se enfoca en enlaces y tareas de Notas.
2. Abriremos una nota y agregaremos una URL sin protocolo para mostrar la normalización.
3. Agregaremos una tarea, guardaremos la nota y volveremos al modo de consulta.
4. Abriremos el enlace y marcaremos la tarea sin entrar nuevamente a Editar.
5. Recargaremos la página para demostrar que los datos permanecen.
6. Mostraremos `note-detail-page.component.ts`, `UpdateNoteHandler` y `schema.prisma`.
7. Explicaremos Repository, Facade, Mapper, DTO, CQRS y los principios SOLID.
8. Cerraremos indicando las mejoras pendientes y los siguientes sprints.

12 Preguntas que podrían hacernos

¿Qué realizaron en el Sprint 1?

Agregamos enlaces URL y tareas persistentes al módulo de Notas, incluyendo interfaz, validación, permisos y base de datos.

¿Dónde se guardan las tareas y los enlaces?

Se guardan en los campos JSON `tasks` y `links` del modelo `Note` en MySQL mediante Prisma.

¿Qué lenguajes utilizaron?

Utilizamos principalmente TypeScript en Angular y NestJS. También utilizamos HTML, SCSS, JSON y el esquema de Prisma para MySQL.

¿Qué patrones de diseño utilizaron?

Se utilizaron CQRS, Repository, Facade, Mapper y DTO. Repository es uno de los más importantes porque separa el caso de uso de Prisma.

¿Dónde aplicaron SOLID?

En la separación Component, Facade, API, Handler, Repository y Mapper, y en la dependencia del Handler hacia el contrato NoteRepository.

¿Cómo controlaron los permisos?

El usuario llega desde el JWT y el Handler comprueba si es propietario o miembro autorizado del repositorio.

¿Qué falta mejorar?

Pruebas automatizadas, tareas durante la creación inicial y la reducción del SCSS que supera el presupuesto.